OPERATING SYSTEMS

Memory Management

Strategies, virtual memory, Linux, and paging on ARM Cortex-A in 32-bit and 64-bit

1 Strategies · 2 Paging and hardware · 3 Virtual memory · 4 Linux · 5 ARM Cortex-A

OVERVIEW

What this covers

Part	Topic	The question it answers
1	Memory management strategies	How can the OS place several processes in physical memory?
2	Paging and hardware support	How do the MMU, page tables and the TLB translate addresses?
3	Virtual memory	How can a process run when it does not fit in RAM?
4	Memory management in Linux	How does Linux organise address spaces, frames and reclaim?
5	Paging on ARM Cortex-A	How do 32-bit and 64-bit ARM cores implement all of the above?

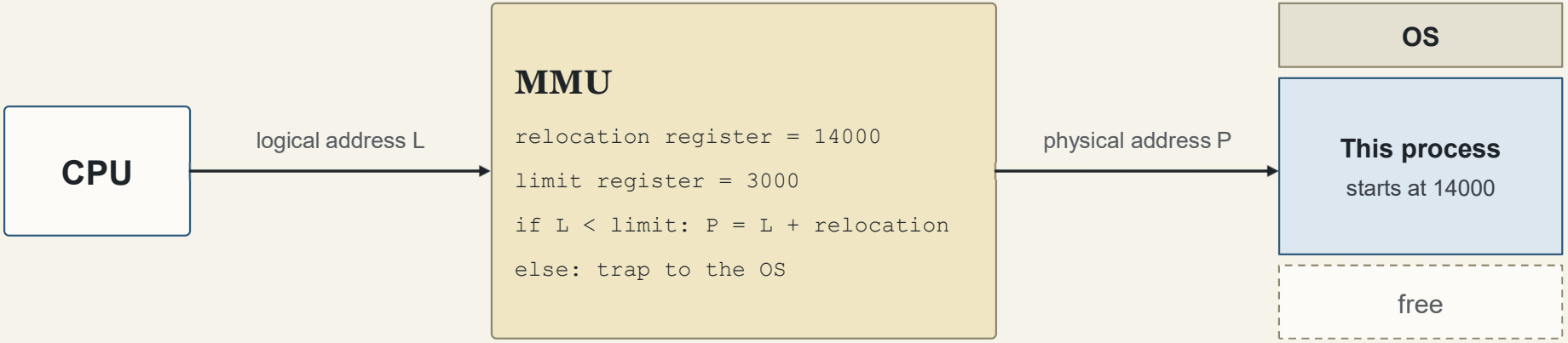
1 · STRATEGIES

What the OS must do with memory

Requirement	What it means	Typical mechanism
Relocation	Load a program anywhere in RAM	Base register, page table
Protection	Keep processes apart from each other and from the kernel	Limit check, permission bits
Sharing	Let several processes use the same code or data	Shared pages and libraries
Logical organisation	Present modules, stack and heap rather than raw bytes	Segments, VMAs
Physical organisation	Move data between small, fast RAM and large, slow storage	Swapping, demand paging

Every strategy that follows is a different answer to these same five requirements.

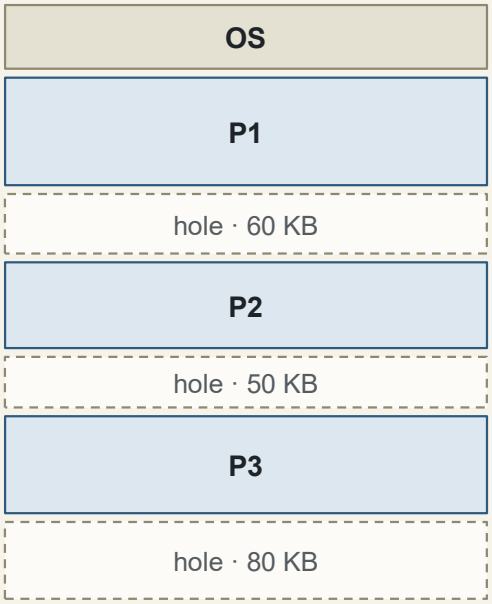
Logical and physical addresses



The relocation register lets the program sit anywhere in RAM; the limit register stops it reaching anything outside its own region.

1 · STRATEGIES

Contiguous allocation and fragmentation



Placement policies

- **First fit**: take the first hole that is large enough
- **Best fit**: take the smallest hole that is large enough
- **Worst fit**: take the largest hole

Fragmentation

- **External**: enough free memory in total, but split into holes that are each too small. A 100 KB request fails here although 190 KB is free.
- **Internal**: a block is larger than the request and the rest of it is wasted
- **Compaction** merges the holes, but only works if programs can be relocated at run time
- With first fit, roughly one third of memory can end up unusable (the 50-percent rule)

1 · STRATEGIES

Segmentation

Segment table

seg	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

Logical address = (segment, offset)

```
(2, 53) → 4300 + 53 = 4353
(3, 852) → 3200 + 852 = 4052
(0, 1222) → 1222 ≥ 1000, trap
```

- Segments follow how programmers think: code, data, stack, each with its own size and permissions
- Sizes vary, so external fragmentation comes back
- Sharing and protection work naturally per segment
- Linux and ARM use paging only; x86 keeps its segments flat for compatibility

1 · STRATEGIES

Comparing the strategies

Scheme	Unit allocated	Fragmentation	Sharing and protection	Where you meet it
Contiguous partitions	Whole process	External, and internal if partitions are fixed	One base and limit per process	MMU-less microcontrollers, early systems
Segmentation	Variable-size segment	External	Per segment	Mostly historical; flat on x86-64
Paging	Fixed-size page	Internal, in the last page only	Per page, through bits in the PTE	Linux, Windows, macOS, Android, RTOSes with an MMU

Paging removes external fragmentation, which is why the rest of the lecture is about paging.

2 · PAGING AND HARDWARE SUPPORT

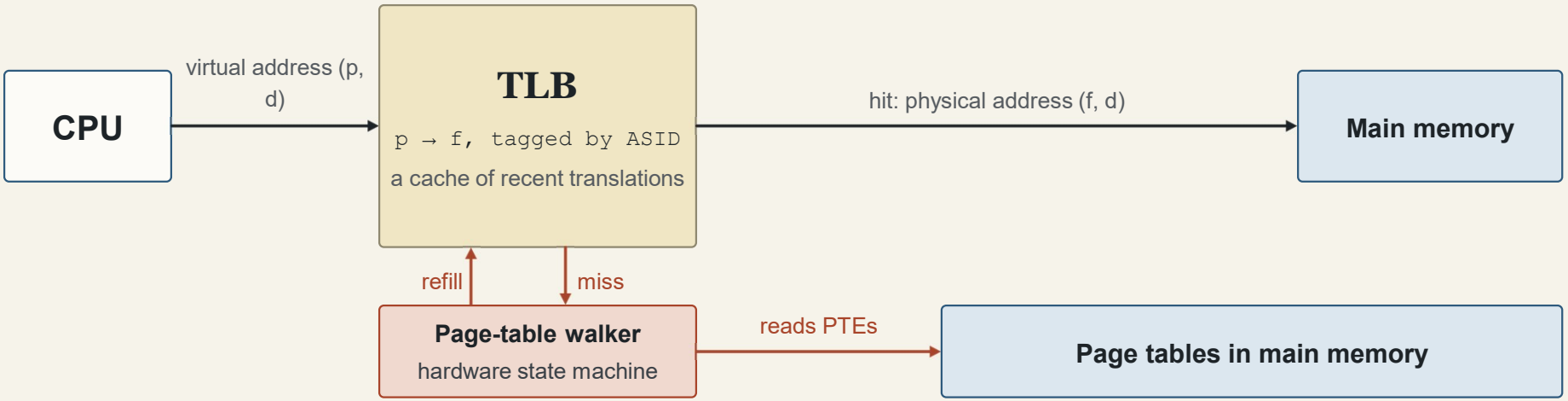
Paging

A logical address splits into a page number p and an offset d. The page table maps p to a frame f, and the physical address is f followed by the same d.

What a page table entry holds

Field	Purpose
Frame number	Which physical frame holds the page
Present (valid)	If clear, any access traps to the OS as a page fault
Read / write	A write to a read-only page traps, which is how copy-on-write works
User / supervisor	Whether unprivileged code may touch the page
Execute never	Stops data pages from being run as code
Accessed	Set by hardware on use, read by the page-replacement code
Dirty	Set on write; the page must be written back before its frame is reused
Cache attributes	Cacheable, device or strongly ordered memory, plus shareability

The MMU and the TLB



Effective access time

```
EAT = h × (t_tlb + t_mem) + (1 − h) × (t_tlb + k × t_mem)
```

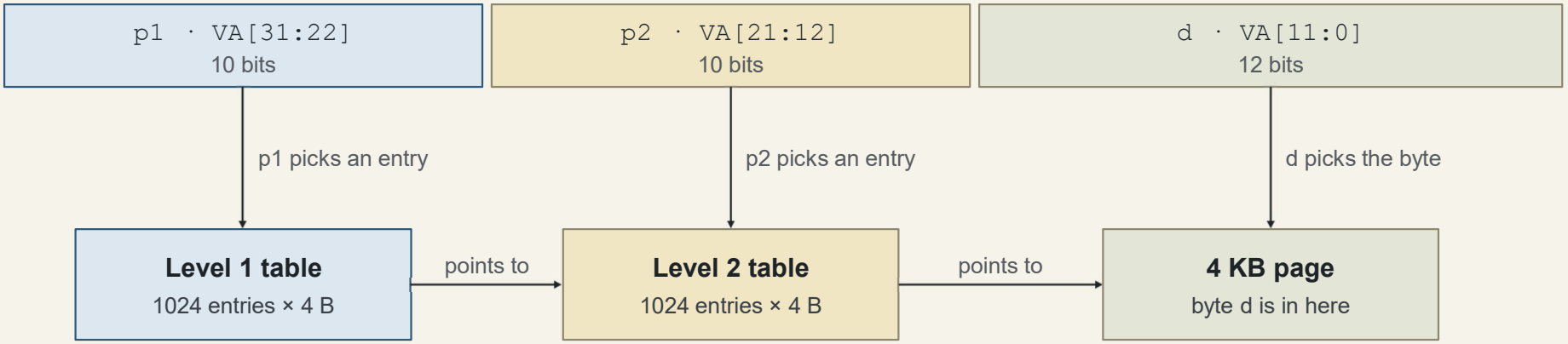
h is the TLB hit ratio, t_tlb = 10 ns, t_mem = 100 ns, and k is the number of page-table levels plus one for the data access itself.

TLB hit ratio	One-level table (k = 2)	Two-level table (k = 3)
80%	130 ns	150 ns
98%	112 ns	114 ns
99.9%	110.1 ns	110.2 ns

Locality of reference keeps hit ratios high, so translation adds only a few percent.

Page tables for large address spaces

One flat table for a 32-bit space with 4 KB pages: 1,048,576 entries × 4 B = 4 MB per process, almost all of it unused.



Regions that are never touched need no level-2 table at all. 64-bit address spaces add more levels, or use hashed or inverted page tables.

3 · VIRTUAL MEMORY

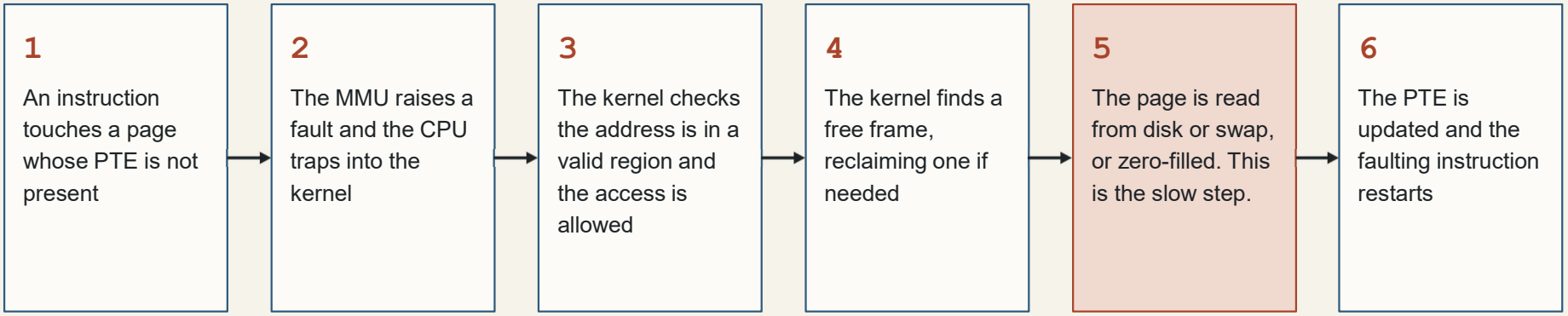
Virtual memory

Each process sees its own large, contiguous address space. The OS backs only the parts in use with RAM and keeps the rest on disk.

1. Programs larger than physical memory can run
2. Isolation: one process cannot even name another’s memory
3. Sharing: libraries and files map into many address spaces
4. Lazy allocation: pages appear on first touch
5. Every program can use the same layout, which simplifies linking and loading

The backing store is swap space, plus the program and library files themselves.

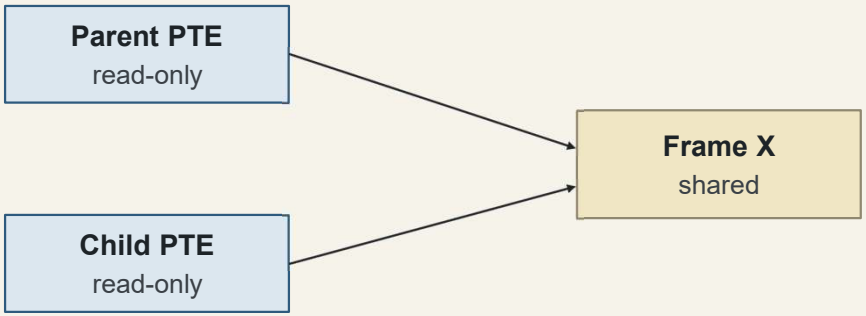
Demand paging and the page fault



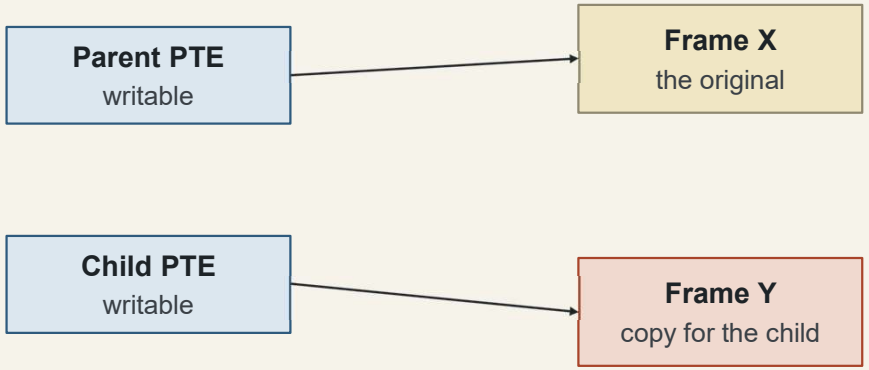
At 200 ns per access and 8 ms per fault, one fault in 1,000 accesses gives 8,199.8 ns per access, about 40 times slower. Keeping the slowdown under 10% allows fewer than one fault in 400,000 accesses.

Copy-on-write

After fork()



After the child writes



fork() copies the page tables, not the pages. The first write takes a protection fault, and only that one page is copied. Lazy zero pages and private file mappings use the same trick.

Page replacement

Algorithm	Rule and trade-off
FIFO	Evict the oldest page. Simple, but more frames can give more faults (Belady's anomaly)
Optimal	Evict the page not needed for the longest time. Impossible in practice, used as a yardstick
LRU	Evict the least recently used page. Good results, costly to track exactly
Clock	Approximates LRU with a reference bit and a circular list; the basis of real kernels

Reference string, 3 frames

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

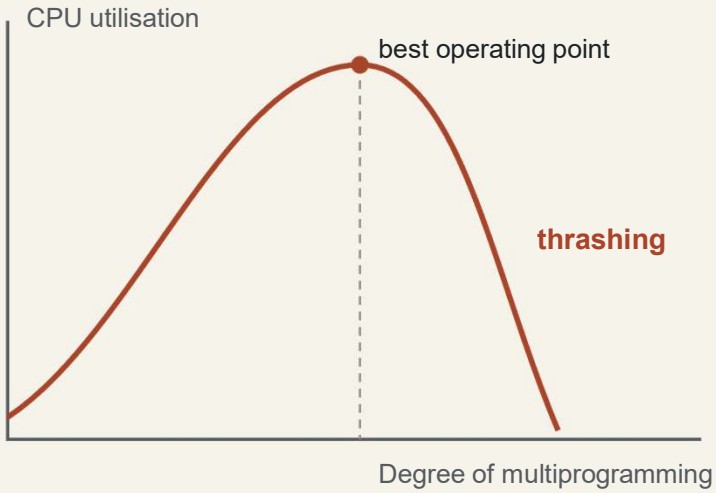
Page faults

15 FIFO

12 LRU

9 Optimal

Thrashing and the working set



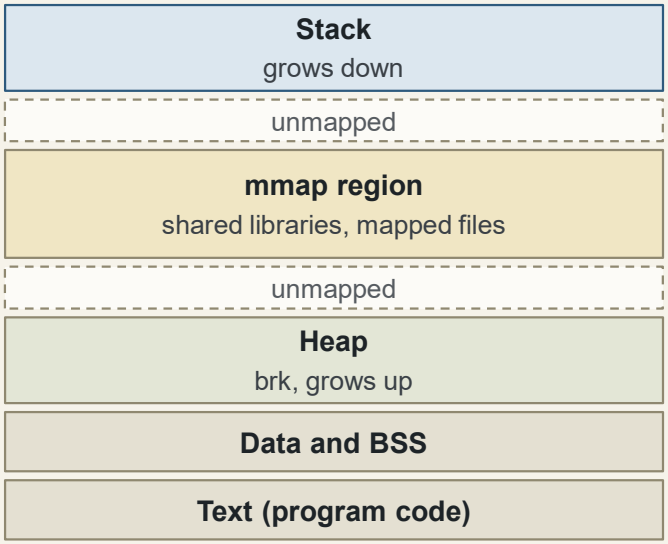
Thrashing: processes spend more time paging than executing. The OS sees an idle CPU and admits more processes, which makes it worse.

Working set: the pages a process used in its last Δ references. Keep each working set resident, or suspend the process.

Page-fault frequency: give a process more frames when its fault rate is too high, and take frames away when it is too low.

Linux has no strict working-set model. It relies on LRU lists, memory cgroup limits, and as a last resort the OOM killer.

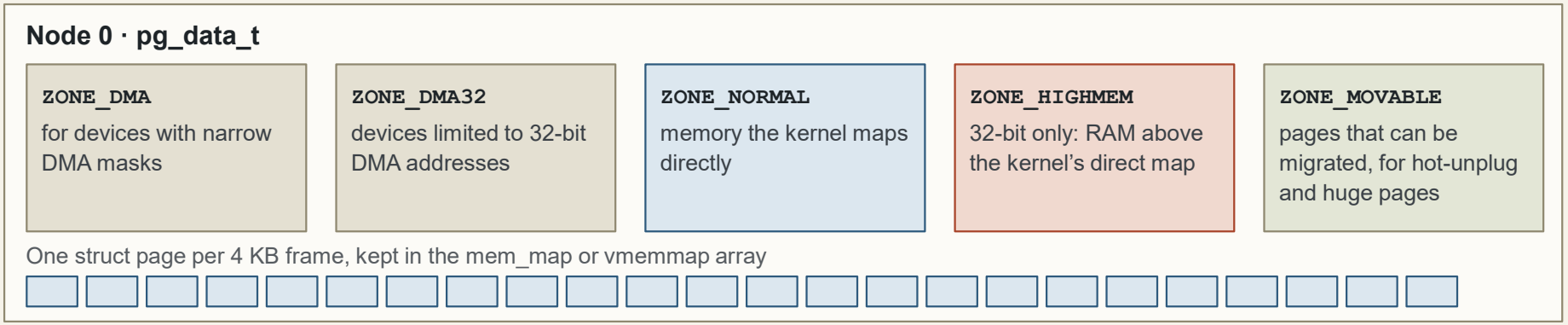
A process address space in Linux



High addresses at the top.

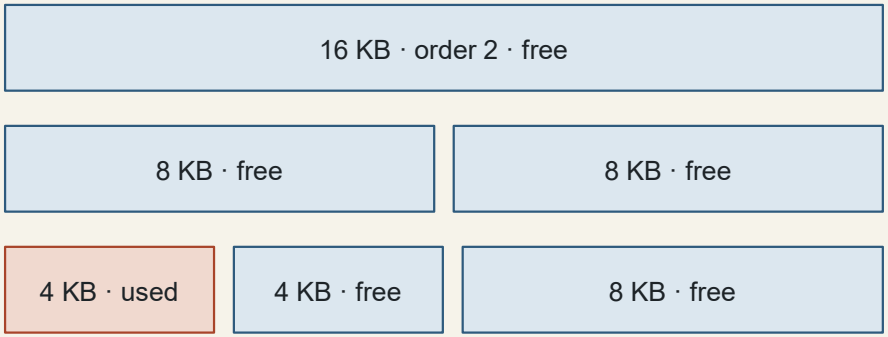
- **mm_struct**: one per process. It holds the page-table root (the pgd), the VMA tree and memory counters.
- **vm_area_struct** (a VMA): start, end, permissions, and the backing file or anonymous memory.
- Page tables fill in lazily, one page fault at a time.
- cat /proc/self/maps lists the VMAs of a process.
- Since Linux 6.1 the VMAs live in a maple tree; earlier kernels used a red-black tree.

Physical memory: nodes, zones and pages



ARM32 typically has Normal and HighMem. arm64 typically has DMA, DMA32 and Normal. Each zone keeps buddy free lists, per-CPU page lists, and min, low and high watermarks.

The buddy allocator



1 A free block of order 2 (16 KB)

2 Split into two buddies of order 1

3 Split once more. One 4 KB page is handed out; its buddy and the other 8 KB stay on the free lists.

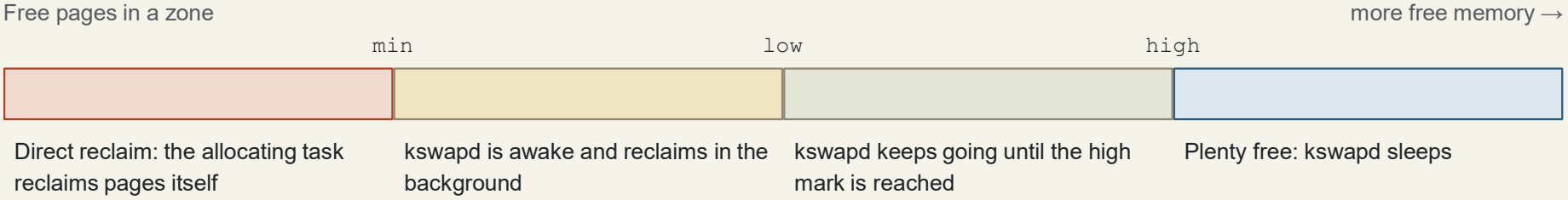
Orders run from 0 (one 4 KB page) to 10 (4 MB). A freed block merges with its buddy when the buddy is also free; the two addresses differ by one bit. /proc/buddyinfo shows the free blocks per order for every zone.

Allocating memory inside the kernel

API	What you get	Typical use
alloc_pages	Physically contiguous pages	Page tables, page-sized buffers
kmalloc	Physically contiguous bytes	Small kernel buffers
kmem_cache_alloc	Physically contiguous objects	Many equal-sized structures
vmalloc	Virtually contiguous only	Large buffers
dma_alloc_coherent	Memory a device can access coherently	Device DMA buffers
ioremap	Maps device registers (MMIO)	Register access in drivers

kmalloc and kmem_cache_alloc sit on top of the slab (SLUB) allocator, which carves pages from the buddy allocator into fixed-size objects. When a device will touch the buffer, use the DMA API rather than handing it a physical address by hand.

Page cache, reclaim and swap



Reclaim order

- Clean page-cache pages: drop them
- Dirty page-cache pages: write back, then drop
- Anonymous pages: write to swap, then free
- Slab caches: shrinkers release objects
- Pages sit on active and inactive LRU lists

Pressure and last resort

- vm.swappiness weighs anonymous against file reclaim
- zram, compressed swap in RAM, is common on boards
- Memory cgroups reclaim and OOM-kill per group
- The OOM killer picks a victim by oom_score

What Linux does on a page fault

Situation	Handler	Result
First touch of an anonymous page	do_anonymous_page	A zero-filled frame is mapped
File-backed page not in memory	do_read_fault	The page is read into the page cache and mapped
Write to a private or post-fork page	do_cow_fault, do_wp_page	The page is copied; the writer gets its own frame
PTE holds a swap entry	do_swap_page	The page is read back from swap
Address outside every VMA, or a permission violation	none	The process receives SIGSEGV

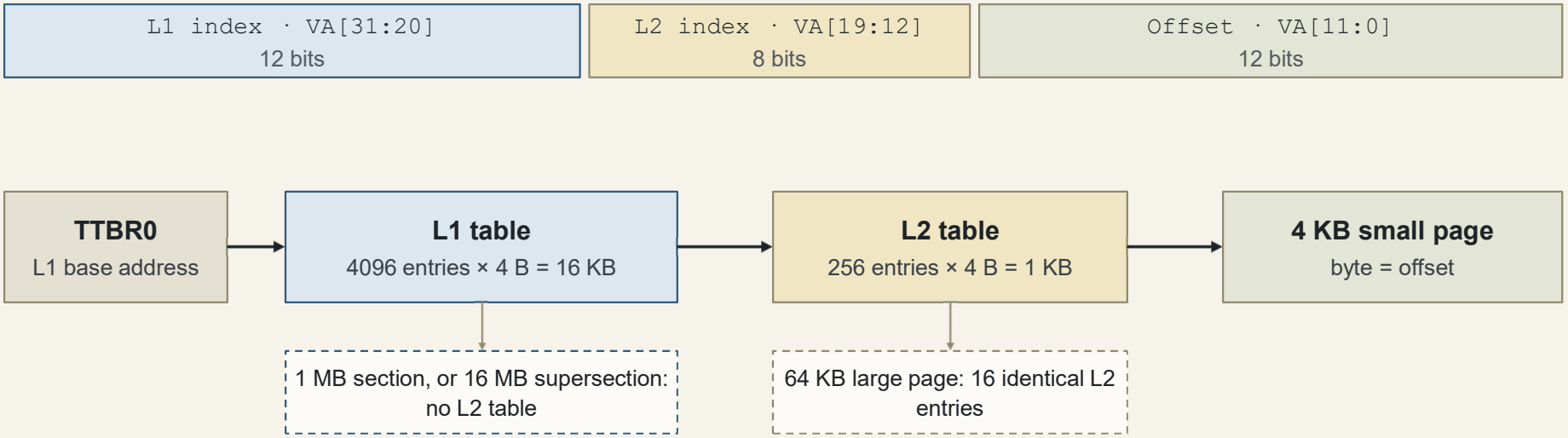
Entry path: the ARM abort handler (do_page_fault in arch/arm/mm and arch/arm64/mm) calls handle_mm_fault(), which reaches handle_pte_fault() and dispatches to the handlers above.

Cortex-A cores and their MMUs

Architecture	Cortex-A cores	Execution state and translation formats
ARMv7-A	A5, A7, A8, A9, A15, A17	AArch32 only. Short-descriptor format; LPAE as well on A7, A15 and A17
ARMv8-A	A53, A55, A57, A72, A76 and later	AArch64, and AArch32 on most. AArch64 translation with 4, 16 or 64 KB granules
ARMv9-A	A510, A710 and later	The same AArch64 translation scheme, with extensions

- A micro-TLB per access port plus a unified main TLB; table walks are done by hardware
- ASIDs tag TLB entries, so a context switch does not need a full TLB flush
- Cores with the virtualization extensions add a second translation stage for hypervisors

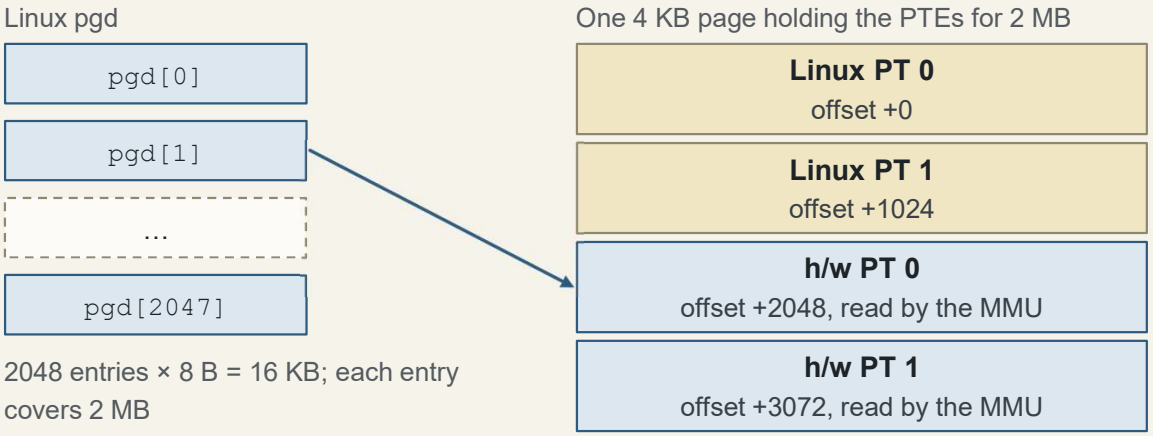
ARMv7-A short-descriptor walk (32-bit)



TTBCR.N splits the address space between TTBR0 and TTBR1, DACR holds 16 domain access fields, and the ASID is in CONTEXTIDR.

ARMv7-A descriptors: what is in an entry

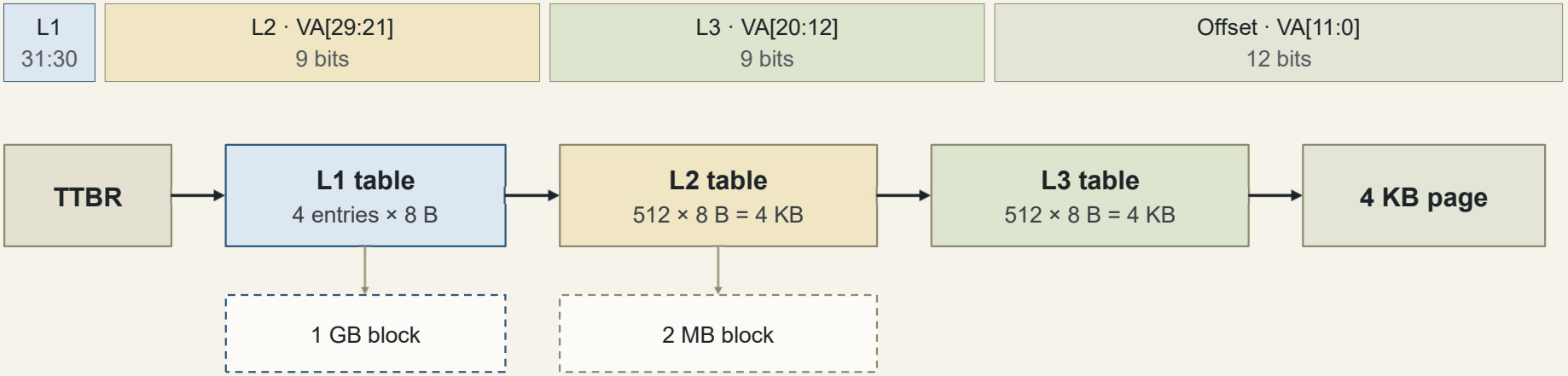
Linux on ARM32: pgd and the shadow tables



2048 entries × 8 B = 16 KB; each entry covers 2 MB

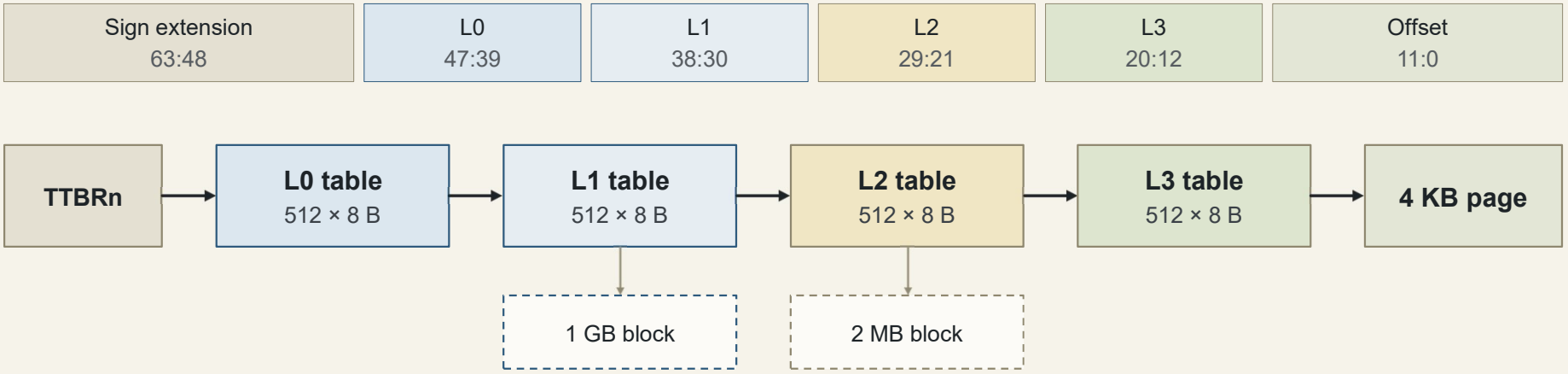
- Each 8-byte pgd slot holds two 4-byte hardware L1 entries
- Hardware PTEs have no accessed or dirty bit, so Linux keeps them in the Linux copy and syncs the two
- User space gets 3 GB, the kernel 1 GB; kernel mappings are copied into every pgd

ARMv7-A with LPAE: three levels, 40-bit PA



- 64-bit descriptors with up to 40-bit physical addresses, so more than 4 GB of RAM is usable
- Hardware accessed flag and no domains; needed for KVM on ARM32 and present on Cortex-A7, A15 and A17

AArch64 walk: 4 KB granule, 48-bit VA



- Each table is 512 entries × 8 B = 4 KB, so every level takes 9 address bits
- Bit 63:48 all 0 selects TTBR0_EL1 (user); all 1 selects TTBR1_EL1 (kernel)

5 · PAGING ON ARM CORTEX-A

Three granule sizes, three table shapes

Granule	Offset bits	Bits per level	Levels for a 48-bit VA	Block sizes
4 KB	12	9	4 (L0 to L3)	1 GB (L1), 2 MB (L2)
16 KB	14	11	4 (L0 has 1 bit)	32 MB (L2)
64 KB	16	13	3 (L1 to L3)	512 MB (L2)

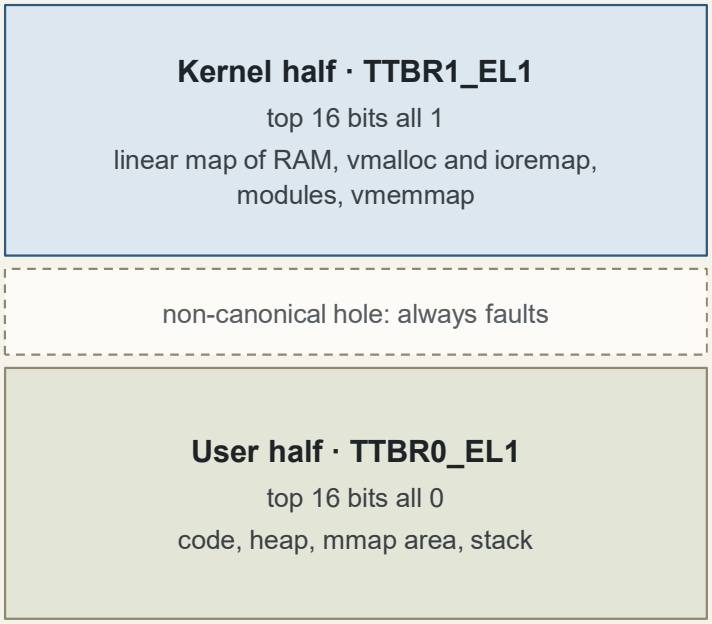
- TTBR0_EL1 and TTBR1_EL1 hold the table base and the ASID for the user and kernel halves
- TCR_EL1 sets the VA size (64 minus T0SZ or T1SZ), the granule per half (TG0, TG1) and the PA size (IPS)
- MAIR_EL1 holds eight memory-attribute encodings, which descriptors select by index
- SCTLR_EL1.M turns the MMU on; ID_AA64MMFR0_EL1 tells which granules the core supports

AArch64 L3 page descriptor (64 bits)



- Bits [1:0] = 11 mean a page at L3, or a table pointer at L0 to L2; 01 is a block
- AP gives read and write rights, UXN and PXN forbid execution at EL0 and at EL1
- AF is the accessed flag; DBM lets hardware track dirty state (ARMv8.1 and later)
- AttrIndx selects one of eight memory types in MAIR_EL1; SH sets shareability
- nG makes the mapping non-global, so the TLB tags it with the ASID

Linux on AArch64: two halves, one kernel map



- Default build: 4 KB pages, 4 levels, 48-bit VA; 39-bit with 3 levels and 52-bit are options
- Linux level names are pgd, p4d (folded), pud, pmd and pte
- A context switch writes TTBR0_EL1 and the ASID; the kernel half in TTBR1 stays put, so there is no per-process kernel copy as on ARM32
- 16-bit ASIDs where supported, allocated in arch/arm64/mm/context.c
- Hardware accessed and dirty tracking when the core has it
- Huge pages of 2 MB and 1 GB, and contiguous-bit groups of entries

The three formats side by side

Feature	ARMv7-A short	ARMv7-A LPAE	AArch64 (4 KB)
Levels	2	3	4
Address size	VA 32, PA 32	VA 32, PA 40	VA 48, PA up to 48
Entry size	4 B	8 B	8 B
Mapping sizes	4 KB, 64 KB, 1 MB, 16 MB	4 KB, 2 MB, 1 GB	4 KB, 2 MB, 1 GB (contiguous hints: 64 KB, 32 MB)
Control registers	TTBR0, TTBR1, TTBCR, DACR	TTBR0, TTBR1, TTBCR	TTBR0_EL1, TTBR1_EL1, TCR_EL1, MAIR_EL1
Linux levels	pgd, pte (pmd folded)	pgd, pmd, pte	pgd, pud, pmd, pte

WRAP-UP

Looking at all this on a running system

Command	Shows
getconf PAGESIZE	The page size in bytes
cat /proc/self/maps	The VMAs of a process
cat /proc/PID/smaps	Per-VMA resident, shared and swapped sizes
cat /proc/buddyinfo	Free blocks per order, per zone
cat /proc/zoneinfo	Zone sizes, watermarks, free pages
cat /proc/meminfo	System-wide totals, cache, swap, slab
cat /proc/vmstat	Counters such as pgfault and pgmajfault
slabtop	Slab caches and their object counts
/proc/PID/pagemap	Virtual-to-frame mapping per page (frame numbers need root)

WRAP-UP

Summary

1. Contiguous schemes fragment; paging removes the problem
2. The MMU and TLB do the translation, and multilevel tables keep the tables small
3. Virtual memory adds demand paging, copy-on-write and page replacement
4. Linux builds on VMAs, zones, the buddy allocator and reclaim
5. ARM32 uses 2 levels, or 3 with LPAE; AArch64 uses 4 levels with 4, 16 or 64 KB granules, and Linux layers its own tables on top